

Gluwa NFT Contract Security Audit

: Gluwa NFT Contract Security Audit

March 11, 2026

Revision 1.0

ChainLight@Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

Gluwa NFT Contract Security Audit	1
Table of Contents	2
Executive Summary	4
Audit Overview	5
Scope	5
Code Revision	5
Severity Categories	6
Status Categories	8
Finding Breakdown by Severity	9
Findings	10
Summary	10
#1 GLUWANFT-001 transferFrom() allows arbitrary token transfers without approval	11
#2 GLUWANFT-002 Inefficient token ownership tracking	14
#3 GLUWANFT-003 totalSupply is not decreased on NFT burn	15
#4 GLUWANFT-004 Minor Suggestions	17
Revision History	18

Executive Summary

From November 14 to November 15, 2025, Theori's ChainLight team conducted a 2-day security audit of Gluwa's NFT contract. The assessment focused on an upgradeable ERC-721-based NFT system with artifact randomization logic, in which the contract supports minting based on registered artifact types, tracks per-user token ownership, and allows administrative control over ownership and randomization parameters.

The review covered core ERC-721 functionality, including transfer authorization, token ownership tracking, minting and burning flows, total supply accounting, artifact selection logic, active artifact management, and ownership transfer safety. During the audit, several issues were identified, including a critical authorization bypass in `transferFrom()` that could allow arbitrary NFT transfers without approval, inaccurate `totalSupply` accounting on burn, inefficient owner token tracking and active artifact enumeration, the use of `OwnableUpgradeable` instead of `Ownable2StepUpgradeable` for ownership changes, and missing validation for `chanceOfFullRandom_`.

Audit Overview

Scope

Name	Gluwa NFT Contract Security Audit
Target / Version	Git Repository (gluwa/CTC-NFT): commit 946bb7ae52ccaf6bde2175c73114f555d7f5bceb
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	1	GLUWANFT-001
High	0	N/A
Medium	0	N/A
Low	1	GLUWANFT-003
Informational	2	- GLUWANFT-002 - GLUWANFT-004
Note	0	N/A

Findings

Summary

#	ID	Title	Severity	Status
1	GLUWANFT-001	<code>transferFrom()</code> allows arbitrary to ken transfers without approval	Critical	Patched
2	GLUWANFT-002	Inefficient token ownership tracking	Informational	Patched
3	GLUWANFT-003	<code>totalSupply</code> is not decreased on N FT burn	Low	Patched
4	GLUWANFT-004	Minor Suggestions	Informational	Patched

#1 GLUWANFT-001 `transferFrom()` allows arbitrary token transfers without approval

ID	Summary	Severity
GLUWANFT-001	The <code>ERC721Upgradeable.transferFrom</code> implementation deviates from the standard ERC-721 logic by calling <code>_transfer()</code> directly. Since <code>_transfer()</code> internally invokes <code>_update()</code> with <code>auth</code> set to <code>address(0)</code> , the authorization check in <code>_checkAuthorized()</code> is skipped. As a result, any arbitrary user can transfer any existing <code>tokenId</code> through <code>transferFrom()</code> without being the owner or an approved operator.	Critical

Description

In the current implementation, `transferFrom()` is defined as follows:

```
function transferFrom(address from, address to, uint256 tokenId) public virtual {
    _transfer(from, to, tokenId);
}
```

Unlike the standard OpenZeppelin ERC-721 implementation, this function does not pass `_msgSender()` as the authorization argument. Instead, it delegates to `_transfer()`, which then calls:

```
address previousOwner = _update(to, tokenId, address(0));
```

Within `_update()`, the authorization check is performed only when `auth != address(0)`:

```
if (auth != address(0)) {
    _checkAuthorized(from, auth, tokenId);
}
```

```
}
```

Because `auth` is set to `address(0)`, `_checkAuthorized()` is never executed. Consequently, `transferFrom()` does not verify whether the caller is the token owner, an approved address, or an authorized operator. This allows any arbitrary caller to transfer any token as long as the supplied `from` address matches the current owner.

Impact

Critical

A malicious user can transfer arbitrary NFTs without approval by directly calling `transferFrom()`. This completely breaks the core ERC-721 authorization model and can result in unauthorized transfer or theft of all minted tokens.

Recommendation

It is recommended to align `transferFrom()` with the standard ERC-721 implementation by passing `_msgSender()` to `_update()` so that authorization is properly enforced.

Remediation

Patched

commit: `bb1237e9ac424fbcf4af6b44b75ef611a391d91f`

#2 GLUWANFT-002 Inefficient token ownership tracking

ID	Summary	Severity
GLUWANFT-002	ERC721Upgradeable maintains a custom <code>_owned</code> array to expose the list of token IDs held by each user via <code>owned()</code> . However, updating this array requires $O(n)$ removal on transfers, which is inefficient and non-standard.	Informational

Description

The contract introduces `ERC721Storage._owned` to track per-user token IDs. During transfers, the token ID is removed from the sender's array via `removeUint256()` and added to the recipient's array. Since removal from a dynamic array requires a linear search, transfer costs grow with the number of tokens owned by the sender.

Impact

Informational

This may cause unnecessary gas overhead for transfers involving users holding many NFTs and reduces scalability compared to standard enumeration approaches.

Recommendation

If per-owner token enumeration is required, it is recommended to use `ERC721EnumerableUpgradeable` or another standardized indexing approach instead of manually maintaining `_owned` with linear-time removals.

Remediation

Patched

commit: `bb1237e9ac424fbcf4af6b44b75ef611a391d91f`

#3 GLUWANFT-003 `totalSupply` is not decreased on NFT burn

ID	Summary	Severity
GLUWANFT-003	In <code>ERC721Upgradeable</code> , <code>totalSupply</code> is increased during minting but is not decreased during burning, causing the reported supply to become inaccurate after tokens are burned.	Low

Description

The contract increments `_totalSupply` in `_update()` when a token is minted:

```
} else {  
    $_totalSupply += 1;  
}
```

However, when a token is burned, `_update(address(0), tokenId, address(0))` only decreases the owner's balance and clears ownership, but does not decrease `_totalSupply`. As a result, `totalSupply()` continues to include burned tokens.

Impact

Low

`totalSupply()` may return an incorrect value after burns, which can break assumptions in integrations, frontends, or accounting logic relying on an accurate circulating NFT count.

Recommendation

It is recommended to decrease `_totalSupply` when a token is burned so that `totalSupply()` remains consistent with the actual number of existing tokens.

Remediation

Patched

commit: `bb1237e9ac424fbcf4af6b44b75ef611a391d91f`

#4 GLUWANFT-004 Minor Suggestions

ID	Summary	Severity
GLUWANFT-004	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, and improving code maturity and readability.	Informational

Description

1. In `ArtifactRandomizer._getArtifactForMinting`, reconstructing the list of active artifacts by iterating over all artifacts on each call is gas-inefficient. It is recommended to maintain the list of active artifacts in a dedicated state variable.
2. It is recommended for `GluwaCTCNFT` to inherit from `Ownable2StepUpgradeable` instead of `OwnableUpgradeable` to make ownership transfers safer.
3. It is recommended to add a check in `ArtifactRandomizer._updateChanceOfFullRandom` to ensure that `chanceOfFullRandom_` is less than `100`.

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Patched

All issues have been patched as recommended.

Revision History

Version	Date	Description
1.0	March 11, 2026	Initial version

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

